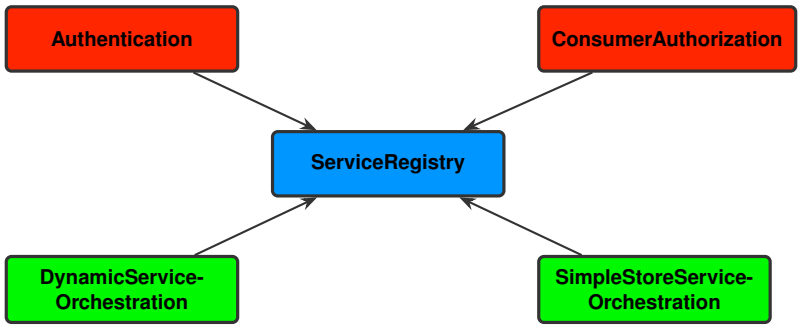
Arrowhead Core Systems 5.2.2 – White Box

Generic System of Systems Design Description



Abstract

This document is the white-box Generic System of Systems Design Description (GSoSDD) of the Eclipse Arrowhead 5 Core Systems as released in version 5.2.2. It complements the black-box, technology-agnostic *GSoSD Arrowhead Core and Support Systems 5.2.2* [1] (successor of *GSoSD Arrowhead Core Systems 5.0* [2]) by describing how the five released Core microsystems – **ServiceRegistry**, **Authentication**, **Consumer Authorization**, **DynamicServiceOrchestration** and **SimpleStoreServiceOrchestration** – are actually designed, configured and interconnected in the reference implementation (`ah5-core-java-spring` v5.2.2 on `ah5-common-java-spring` 5.2.2), including their internal processing chains, data stores, produced and consumed services, security mechanisms, and the optional interaction with the 5.2.2 Support Systems **Blacklist**, **TranslationManager** and **DeviceQoSEvaluator**.

Contents

1 Overview **4**
1.1 Implementation baseline and roadmap status (30 October 2026) . . . 4
1.2 Core implementation and local-cloud lifecycle updates . . . 5
1.3 Significant Prior Art . . . 7
1.4 How This SoS Is Meant to Be Used . . . 7
1.5 SoS Functionalities and Properties . . . 8
1.6 Important Delimitations . . . 10

2 SoS Architecture **11**
2.1 Constituent Microsystems . . . 11
2.2 Common Internal Architecture of a Core Microsystem . . . 11
2.3 Request Filter Chain . . . 12
2.4 Local Cloud Bootstrapping . . . 12
2.5 Deployed Java Reference Cloud (24 October 2026) . . . 14
2.6 Core Control Flow in the Deployed Cloud . . . 15

3 Microsystem White-Box Descriptions **16**
3.1 ServiceRegistry . . . 16
3.2 Authentication . . . 16
3.3 ConsumerAuthorization . . . 17
3.4 DynamicServiceOrchestration . . . 18
3.5 SimpleStoreServiceOrchestration . . . 18
3.6 Interacting Support Systems (5.2.2) . . . 19
3.7 Plant-Managed Dynamic Orchestration (Local Extension) . . . 20

4 Services **20**
4.1 Produced Services . . . 20
4.2 Consumed Services . . . 21

5 Use Cases (White Box) **23**
5.1 Bootstrapping and Service Registration . . . 23
5.2 Dynamic Orchestration with Authorisation and Tokens . . . 23
5.3 Atomic (Simple Store) Orchestration . . . 24
5.4 Protected Management Access . . . 24

6 Control and Data Planes **25**

7 Security **25**
7.1 Default Secure Profile and Development Exceptions . . . 25
7.2 Security Model . . . 25

8 Non-Functional Requirements as Realised **27**

9 Port Numbering Specification **27**
9.1 Principles . . . 27
9.2 Port Block of a Local Cloud . . . 27
9.3 Canonical Port Allocation . . . 28
9.4 Systems of Local Clouds . . . 29
9.5 Allocation Procedure . . . 30
9.6 Conformance of the 5.2.2 Implementations . . . 30

10 Interoperability with the Java Reference Implementation **31**
10.1 Evidence and Current Deployment Scope 31
10.2 Contract and Security Requirements 32
10.3 Boundaries and Acceptance of Further Replacements 32

11 Implementation Observations and Open Issues **33**

12 References **33**

13 Revision History **36**
13.1 Amendments 36
13.2 Quality Assurance 36

1 Overview

Graphics colour convention. All diagrams use the SysML v1.6 Papyrus library's system-role fills, taken from `EclipseArrowhead-5.0/Arrowhead-5.0.notation`: registry blue, authentication and authorisation red-orange, orchestration green, support yellow and application orange. Both orchestration implementations share the library's combined-cloud green; infrastructure is grey and mixed-system groups are neutral. These colours identify roles rather than implementation languages. The library's turquoise is reserved for service-interface elements.



1.1 Implementation baseline and roadmap status (30 October 2026)

The architecture is realised by three primary source distributions: `Eclipse-Arrowhead-java`, `Eclipse-Arrowhead-go` and `Eclipse-Arrowhead-rust`, under the Eclipse workspace. These are the sources for subsequent implementation changes; older `Arrowhead/` and `EA-fork/` checkouts are historical references or deployment consumers. A primary-source designation does not imply that every running image has been rebuilt from that tree. Release v5.2.2, local extensions, independently packaged sources and dated deployment evidence remain distinct [3].

Language	Current source/build boundary	Evidence and limits
Java	Five core systems in `ah5-core-java-spring`; fifteen independent support projects with own JAR/image/configuration. The Java-cloud manager is Node.js/nginx, despite its folder name.	Support packaging verified on 25 October; core/support services and operations tools have separate lifecycles. No shared JavaSupportRuntime dependency in the fifteen standalone projects.
Go	Five independent core modules and eighteen current top-level support/test/tool modules; private helper copies and optional `go.work`.	28 October audit: 22 module builds, tests and vet checks passed. Packaging does not establish complete Java parity or a new live rollout.
Rust	Five core entry points; fifteen support entry points and a separate LocalCloudManager crate in one Cargo workspace. Core/support libraries and messaging remain shared build dependencies.	28 October copy report: 147 workspace tests passed. Per-system folders are not independently copied source closures. Java-equivalence and operational qualification remain open.

Table 1: Current implementation structure, not a language-equivalence claim. Dated evidence is retained in each primary distribution [4, 5, 6].

Core inventory versus default cloud. Five core implementations remain available. A *new Java local-cloud design* selects ServiceRegistry, Authentication, ConsumerAuthorization and DynamicServiceOrchestration. SimpleStoreServiceOrchestration (labelled SimpleOrchestration by the configurator) is optional and absent from that default; it has not been deleted or implicitly stopped in an existing cloud. This is a configuration default, not a removal of the simple-store architecture or a new upstream release requirement [7].

Database ownership and access boundary. Every defined database is an *integrated asset of its owning microsystem*, including a separately deployed SQL or graph-database container. It is not selectable shared infrastructure or an independent service provider. Only host internal actions or services produced/consumed by that host mediate data access. The ownership rule also applies to embedded SQLite and private snapshot files. Brokers, relays and the constrained Docker proxy remain infrastructure. The Java catalog implements this rule for all sixteen defined database services; the default canvas therefore contains four microsystems, each with its own database, and one AMQP broker. The database assets do not become extra graph nodes [8].

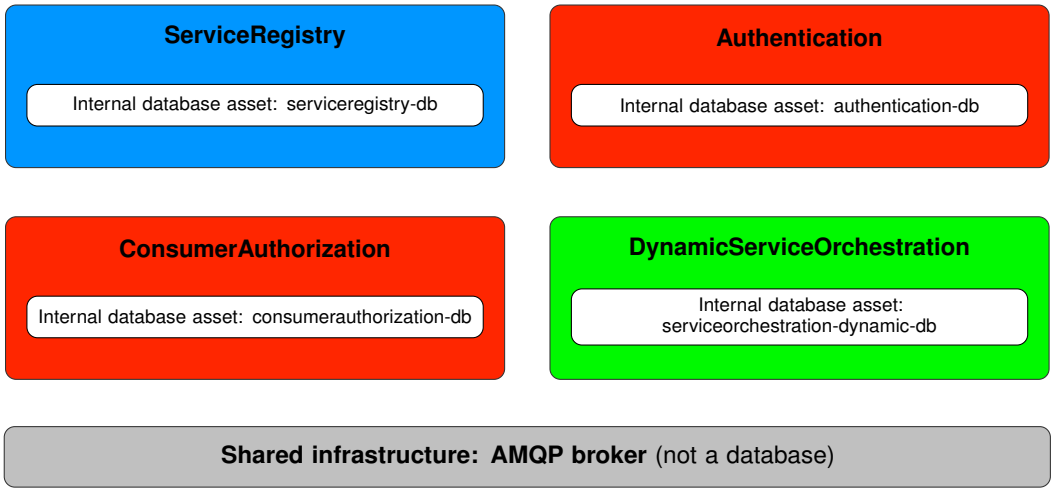


Figure 1: New Java default design. Each database is nested inside its owning microsystem boundary; it is not shared infrastructure or an independently selectable component. Colors retain the established Arrowhead system roles.

Evidence-qualified roadmap. Recorded 19–24 October interoperability and federation results below remain historical acceptance of the stated workloads. The 23 October operation-count audit has not been replaced by a new complete differential audit. The Go packaging audit records a failing optional native-registry integration test: orchestration service registration is rejected, including with unmodified reference sources. Rust still has partial DTO/error/side-effect coverage, unsupported integrations and no automatic Java-state import. Complete replacement requires contract, policy, persistence and mixed-cloud acceptance for the exact image/configuration, beyond successful packaging tests [5, 6, 9].

1.2 Core implementation and local-cloud lifecycle updates

The detailed Java layers and service contracts below remain the reference design. Go modules are separately buildable with `GOWORK=off`; Rust per-system entry points call the shared core library. Private Go helper copies require security fixes in each affected module; Rust library updates require rebuilding affected binaries. Folder symmetry does not mean identical build or persistence models.

Java core SQL containers and their volumes belong inside each core microsystem's white-box boundary. Go/Rust private persistent snapshots are likewise owned state, not services shared with other microsystems. Existing Java SQL state and sessions are not automatically migrated when substituting a Go/Rust binary. The five implementations and their contracts remain documented; selecting the four-system default does not remove the simple-store implementation.

The Java LocalCloudConfigurator catalog groups each host's database and service under one owner. A dependency-ordered plan starts the database before its host; an explicit host stop reverses that order and retains volumes. Independent database stop requests are refused. The executor validates catalog hash, owner, step kind and exact target, refuses host startup without its running database, and refuses database shutdown while the host is running. Each of the sixteen containerised Java-cloud databases has a dedicated internal network shared only with its host, with no published database port; all sixteen boundaries were checked against Compose and Docker runtime on 30 October [8].

Plan review and application remain separate: a fresh runtime plan must match the reviewed hash, and only one active deployment is accepted across API restarts. Core systems, required infrastructure and manager access are protected. The private API uses an operator bearer credential through the manager proxy; it adds no Arrowhead identity or grants. ColonyOS commands are authenticated and catalog checked; operations scripts are not a distributed transaction and have no automatic rollback. The host executor uses an immutable content-addressed JAR and assignment heartbeats preserve API liveness. The verification report records 20 Java tests, ten focused manager/editor tests and ten live browser check groups. Live execution verified the

baseline without mutating microsystems; start/stop paths were unit-tested [8].

The Eclipse Arrowhead project provides a SOA/microservice architecture infrastructure that enables the creation of automation solutions built from loosely coupled, late-bound microsystems which produce and consume microservices from the deep edge to the cloud. The Core microsystems manage the complexity introduced by this distribution by providing *discovery*, *authentication*, *authorisation* and *orchestration* for a local cloud.

The black-box *GSoSD Arrowhead Core and Support Systems 5.2.2* [1], like its predecessor *GSoSD Arrowhead Core Systems 5.0* [2], describes these concerns in the abstract (black box): what problems the Core systems solve and how they interact, without endorsing a specific implementation. The present document is its *white-box* counterpart. It opens the boxes and describes, for the 5.2.2 release of the reference implementation:

- which microsystems realise the abstract Core functions and how they are decomposed internally (layers, filter chains, engines, persistent data);
- the exact microservices, operations, interface templates and default endpoints they produce, and which services each of them consumes;
- the run-time interaction patterns (bootstrapping, discovery, authentication, authorisation, dynamic and store-based orchestration) as realised in code;
- the configuration surface which selects between the alternative behaviours foreseen in the GSoSD (e.g. atomic vs. advanced orchestration, authentication policies, management policies);
- implementation observations relevant to deployers and to the Roadmap WG.

The original reference descriptions were derived from the tagged `v5.2.2` sources of `ah5-core-java-spring` [10], the 5.2.2 `arrowhead-common` library [11], the v5.2.2 support system repositories [12, 13, 14], and the AITIA implementation documentation [15], which holds the per-system SysD/SysDD, SD and IDD documents referenced below. Local extensions and current packaging/status updates are sourced separately in Section 1.1; they are not claims about the upstream tag.

The rest of this document is organised as follows. Section 1.3 references prior art, Section 1.4 the intended usage, Section 1.5 the SoS properties and Section 1.6 the delimitations. Section 2 describes the SoS architecture and the internal architecture common to all Core microsystems. Section 3 gives the white-box description of each microsystem. Section 4 lists produced and consumed services. Section 5 presents white-box use cases. Section 6 discusses control and data planes, Section 7 security, Section 8 non-functional properties, Section 9 specifies port numbering for local clouds and systems of local clouds, and Section 11 lists implementation observations and open issues.

1.3 Significant Prior Art

- **Arrowhead Framework 4.x (last 4.6.1).** Core systems sharing a common SQL database; Service Registry, Authorization and Orchestrator as the mandatory core. The 5.x line breaks backwards compatibility deliberately: every 5.x microsystem owns its own data store.
- **GSoSD Arrowhead Core Systems 5.0 v1.2** [2]. Defines the abstract Core: Service Registry, Authorisation, Authentication and Orchestration systems, the two orchestration approaches (*atomic* and *advanced*), the authorisation priority rule (local cloud Authorisation System rules override application-internal rules), active and passive systems, and the backwards-compatibility policy.
- **Naming convention 2** [16]. Microsystems in PascalCase, microservices (service definitions) in camelCase, operations in kebab-case, interface templates in snake_case. The 5.2.2 implementation enforces this through name normalisers and validators in `arrowhead-common`.
- **Arrowhead X.509 certificate profile.** Hierarchical certificate chain (master → organisation → local cloud → device/system/operator) used by the `CERTIFICATE` authentication policy.

1.4 How This SoS Is Meant to Be Used

A 5.2.2 local cloud follows the v5.0 default of full AA and encrypted communication: ServiceRegistry, Authentication and ConsumerAuthorization provide the protected discovery and AA foundation. Orchestration strategy and optional Support systems are selected for the deployment without weakening that security baseline. Application microsystems (providers and consumers) register themselves and their service instances in the **ServiceRegistry**, obtain an identity either from an X.509 certificate or from the **Authentication** system, and either look up providers directly or ask an orchestration system which provider instances to consume. Providers delegate access decisions to **ConsumerAuthorization**. Figure 2 shows the SoS as a component diagram; solid arrows denote service consumption between Core systems, dashed arrows optional consumption of Support systems.

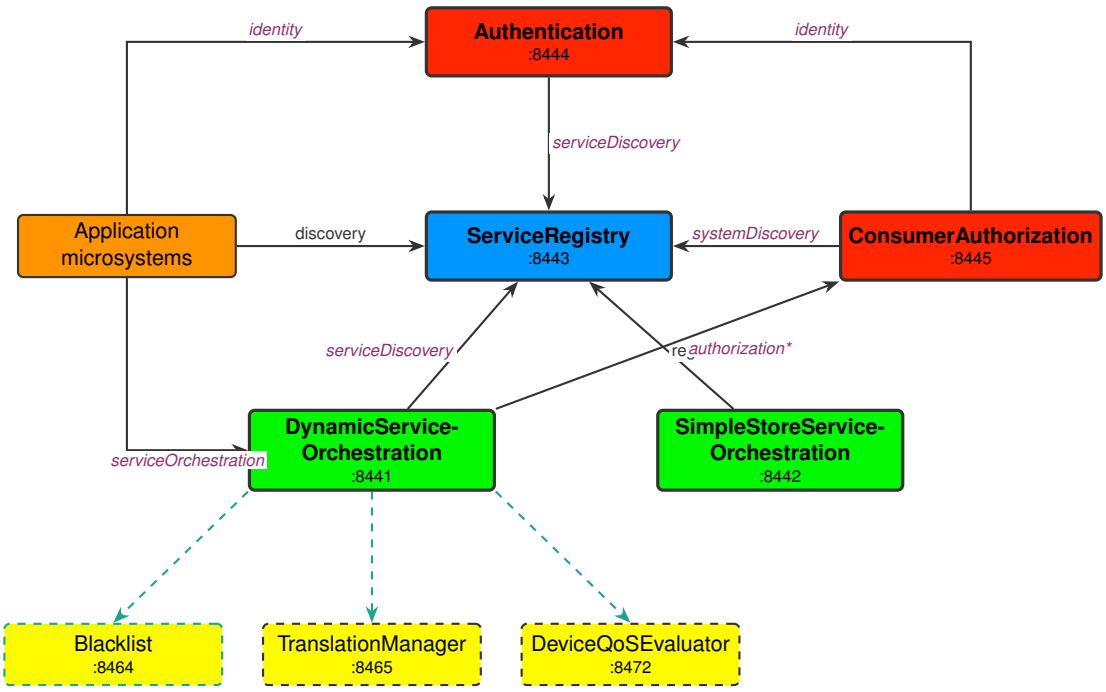


Figure 2: Component view of a 5.2.2 local cloud. Ports are the canonical ports of Section 9. *authorization** denotes the *authorizationManagement* and *authorizationTokenManagement* services. Every Core system additionally consumes *identity* when running with the `OUTSOURCED` authentication policy and *authorization* when its management policy is `AUTHORIZATION`.

Typical deployments are:

Minimal (discovery only). **ServiceRegistry** alone; consumers look up providers directly (requires `service.discovery.policy=open` or listing consumers in `service.discovery.direct.access`).

Atomic orchestration. **ServiceRegistry** + **SimpleStoreServiceOrchestration**; operator-maintained, prioritised consumer→instance rules.

Advanced orchestration. **ServiceRegistry** + **DynamicServiceOrchestration** + **ConsumerAuthorization** (+ **Authentication**); orchestration returns complete access information including authorisation tokens.

Extended. Advanced orchestration plus **Blacklist**, **TranslationManager** and/or **DeviceQoSEvaluator**, enabled per feature flag.

1.5 SoS Functionalities and Properties

1.5.1 Functional properties of the SoS

- **Registry of devices, systems and service instances** with metadata, addresses, versions and typed interfaces (interface templates such as `generic_http`, `generic_https`, `generic_mqtt`, `generic_mqtts`).
- **System identity** through X.509 certificates, self-declaration, or password login to **Authentication** which issues time-limited identity tokens.
- **Two-level access control** (management level overriding provider level) with policy types `ALL`, `WHITELIST`, `BLACKLIST` and `SYS_METADATA`, per service definition or event type and optionally per operation.
- **Access tokens**: time-limited, usage-limited and self-contained tokens (Base64 or RSA-SHA256/512 JSON Web Tokens), plus translation-bridge tokens.
- **Orchestration** in pull and push (subscription) mode, either rule-based from a prioritised store (atomic) or dynamically computed (advanced) with exclusivity locking, preferred providers, matchmaking, authorisation filtering, token issuing, and optional blacklist, QoS, translation and inter-cloud handling.
- **Uniform management**: every system exposes *generalManagement* (`get-log`, `get-config`) and protects all `/mgmt/` endpoints with a configurable management policy.

1.5.2 Configuration of SoS properties

Each microsystem is configured by its own `application.properties` (Spring Boot). The key SoS-level switches are summarised in Table 2.

1.5.3 Data stored by the individual microsystems

In line with the 5.x principle of relative system independence, every Core microsystem owns a private MySQL schema (JPA/Hibernate, schema validated at start-up). No schema is shared.

1.5.4 Non-functional properties

- **Security:** full AA and encrypted communication are the default from v5.0; the reference profile uses mutual TLS, certificate-bound identity tokens, provider/operation authorisation and protected management. Implementation test modes are not equivalent default profiles. Token secrets and provider keys are additionally encrypted at rest in **ConsumerAuthorization**. See Section 7.
- **Latency:** every request passes a filter chain which may call other systems (*identity* `identity-verify` for outsourced authentication, *authorization* `verify` for management requests, *blacklistDiscovery* `check`). Deployers should account for these extra round trips on the control plane.
- **Scalability:** push orchestration is processed asynchronously by a bounded worker pool (`push.orchestration.max.thread`); queries are paginated (`max.page.size`, default 1000).
- **Resource footprint:** Java 21 / Spring Boot 3.4 services with a separately deployed, host-owned MySQL server; intended for fog/edge-server class hardware rather than deep-edge devices.
- **Safety and energy:** no specific mechanisms are implemented.

Property	Secure default profile	Effect
`authentication.policy`	`OUTSOURCED` with mTLS; `CERTIFICATE` for an explicitly certificate-based binding	How a requester's identity is established: `DECLARED`, `CERTIFICATE`, `OUTSOURCED`; **Authentication** itself always uses `INTERNAL`.
`server.ssl.enabled`	`true`; client authentication required	Selects HTTPS/MQTTS and the `generic_https`/`generic_mqtts` interface templates.
`enable.management.filter`, `management.policy`, `management.whitelist`	`true`, `authorization`	Protection of `/mgmt/` endpoints: `SYSOP_ONLY`, `WHITELIST`, `AUTHORIZATION`.
`enable.blacklist.filter`, `force.blacklist.filter`	`false`, `true`	Inbound requester check and orchestration candidate filtering against **Blacklist**.
`service.discovery.policy`, `service.discovery.direct.access`	`restricted`; `DynamicServiceOrchestration`, `TranslationManager`, `DeviceQoSEvaluator`	Whether arbitrary systems may look up arbitrary services in the **Service Registry**, or only those flagged `unrestrictedDiscovery`.
`service.discovery.interface.policy`	`restricted`	Handling of unknown interface templates on registration: `restricted`, `extendable`, `open`.
`enable.authorization`, `enable.translation`, `enable.qos`, `enable.intercloud`	`true`, `false`, `false`, `false`	Feature set of **DynamicServiceOrchestration**.
`mqtt.api.enabled`	`false`	Adds MQTT interfaces to all registered services.
`normalization.mode`	`extended`	Automatic transformation of identifiers to the naming convention.

Table 2: SoS-level switches for the default secure cloud profile. Component sample values are not a statement of the architectural default; disabled security settings are development exceptions. Other feature choices remain deployment-specific.

Microsystem	Schema	Persistent entities
ServiceRegistry	`ah_serviceregistry`	Device, DeviceAddress, System, SystemAddress, DeviceSystemConnector, ServiceDefinition, ServiceInstance, ServiceInstanceInterface, ServiceInterfaceTemplate, ServiceInterfaceTemplateProperty, Logs
Authentication	`ah_authentication`	System, PasswordAuthentication, ActiveSession, Logs
ConsumerAuthorization	`ah_consumer_authorization`	AuthMgmtPolicyHeader, AuthProviderPolicyHeader, AuthPolicy, TokenHeader, TimeLimitedToken, UsageLimitedToken, SelfContainedToken, EncryptionKey, CryptographerAuxiliary, Logs
DynamicService Orchestration	`ah_serviceorchestration_dynamic`	OrchestrationJob, OrchestrationLock, Subscription, QosEvalResult, OrchestrationPlantRules (local extension), Logs
SimpleStoreService Orchestration	`ah_serviceorchestration_simple`	OrchestrationStore, OrchestrationJob, Subscription, Logs

Table 3: Data ownership per Core microsystem.

1.5.5 Stateful or stateless

All Core microsystems are *stateful* with respect to their persistent stores (Table 3). Request handling itself is stateless except for: identity sessions (**Authentication**, expiring after `identity.token.duration`, default 300 s), orchestration locks and push subscriptions (orchestration systems), and usage counters of usage-limited tokens. Scheduled Quartz *cleaner jobs* remove expired sessions, tokens, locks, subscriptions and aged orchestration history.

1.6 Important Delimitations

- The document describes the *tagged v5.2.2 release*. Local, unreleased modifications (e.g. experimental *sysMonitor* telemetry) are out of scope.
- Only the five released Core microsystems are described in detail. **Blacklist**, **TranslationManager** and **DeviceQoSEvaluator** are described only as far as the Core systems interact with them. Other support systems (Gatekeeper/Gateway for inter-cloud, EventHandler, DataManager, PlantDescriptionEngine, . . .) are not part of the 5.2.2 release and are not covered.
- Inter-cloud orchestration is foreseen in the interfaces (`ALLOW_INTERCLOUD`, `ONLY_INTERCLOUD`, `enable.intercloud`) but has no released counterpart system in 5.2.2. Project implementations of **Gatekeeper** and **GateTunnel** provide it over an organisation-level MQTT relay [17]; with them, a consumer's orchestration result is a **GateTunnel** session endpoint in its own cloud.
- Only *system* authentication is supported; user authentication is still under investigation (as stated in [2]). The only **Authentication** method is `PASSWORD`.
- Onboarding and certificate issuing are outside the Core. Without them certificates must be provisioned out of band; with the **Onboarding** and **CertificateAuthority** Support systems they are issued through an operator-controlled onboarding procedure [17].

2 SoS Architecture

2.1 Constituent Microsystems

Table 4 maps the abstract Core systems of [2] to the 5.2.2 microsystems.

GSoSD 5.0 role	5.2.2 microsystem	Module	Port	Approach
Service Registry	**ServiceRegistry**	`serviceregistry`	8443	Bootstrap anchor; must start first
Authentication system	**Authentication**	`authentication`	8444	Password login, identity tokens
Authorisation system	**ConsumerAuthorization**	`consumer-authorization`	8445	Policy engine and token engine
Orchestration system (advanced)	**DynamicServiceOrchestration**	`serviceorchestration-dynamic`	8441	Computed at request time
Orchestration system (atomic)	**SimpleStoreServiceOrchestration**	`serviceorchestration-simple`	8442	Prioritised rule store

Table 4: Mapping of abstract Core roles to 5.2.2 microsystems.

As recommended by [2], a local cloud should run *one* orchestration system. Both orchestration systems produce the same *serviceOrchestration* service definition (same operations `pull`, `subscribe`, `unsubscribe`) and advertise their strategy in the service metadata key `orchestrationStrategy`, so consumers are independent of the chosen approach.

2.2 Common Internal Architecture of a Core Microsystem

All Core microsystems share the same layered design provided by `arrowhead-common` (Figure 3).

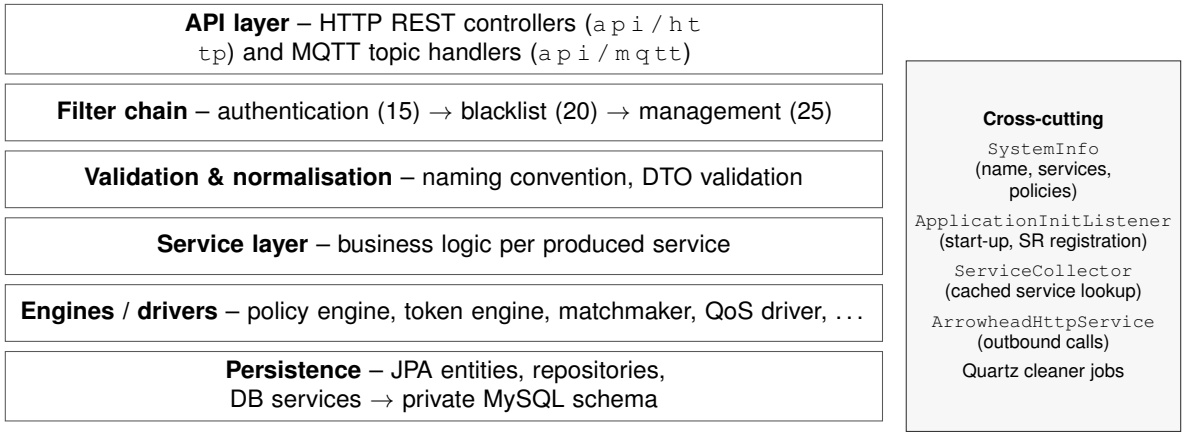


Figure 3: Internal layering shared by all 5.2.2 Core microsystems.

SystemInfo. Each microsystem has one `SystemInfo` bean that defines its system name, its system model (address, version, metadata, X.509 public key when certificate authenticated) and the list of `ServiceModels` it produces. Interface models are generated from the actual configuration: HTTP or HTTPS template, domain name, port, base path and per-operation HTTP method and path; MQTT interfaces with base topics are added when `mqtt.api.enabled=true`.

ApplicationInitListener. At start-up the listener validates the server configuration, loads the key store and checks the server certificate against the Arrowhead certificate profile (secure mode), subscribes to MQTT topics, and – for all systems except **ServiceRegistry** – checks the **ServiceRegistry** connection (3 retries, 15 s apart), revokes any stale registration, registers the system and then each of its services. On shutdown it revokes its services and, if outsourced, logs out.

ServiceCollector. Outbound consumption is name-based: a caller asks for *service definition + operation*; the collector resolves the provider via the **ServiceRegistry** (mode `SR_ONLY`) or via **ServiceRegistry** and orchestration (`SR_AND_ORCH`, default) and caches the resolved model.

Cleaner jobs. Quartz jobs periodically purge expired data (sessions, tokens, locks, subscriptions, history).

2.3 Request Filter Chain

Every inbound HTTP request (and, equivalently, every MQTT message) passes the ordered filter chain of Figure 4 before reaching a controller. The authentication filter sets two request attributes – the *authenticated system name* and the *sysop* flag – on which all later decisions are based.



Figure 4: Ordered inbound filter chain (`arrowhead-common`), reached after TLS peer authentication and decryption. Provider/service authorisation remains required after identity verification; it is not replaced by the management filter.

Authentication filters.

- **SelfDeclaredFilter (`DECLARED`)**: expects `Authorization:BearerSYSTEM//<SystemName>`. The name is trusted as given; the name `Sysop` yields operator rights. Intended for development only.
- **CertificateFilter (`CERTIFICATE`)**: requires mutual TLS. The client certificate must have the `SYSTEM` (`sy`) or `OPERATOR` (`op`) profile type and belong to the same local cloud as the server (compared on the common name). The system name is derived from the CN; an operator certificate yields sysop rights.
- **OutsourcedFilter (`OUTSOURCED`)**: accepts `BearerIDENTITY-TOKEN//<token>`, verified by calling *identity* `identity-verify` on **Authentication**, or `BearerAUTHENTICATOR-KEY//<SystemName>//<HMAC>` for systems sharing a secret key (used by **Authentication** itself). A **ServiceRegistry** lookup of exactly the *identity* service may proceed without an identity token to break the bootstrap cycle; mutual TLS and peer/certificate checks still apply. This is not anonymous or unencrypted access.

Management filter. Requests whose path contains `/mgmt/` are allowed according to `management.policy`: `SYSOP_ONLY` (operator only), `WHITELIST` (operator or listed system), or `AUTHORIZATION` (operator, listed system, or a positive *authorization* `verify` answer from **ConsumerAuthorization** for the service definition and operation that the path and method map to).

2.4 Local Cloud Bootstrapping

The v5.0-and-later baseline is full authentication and authorisation (AA) with encryption of exchanged information. The reference implementation must establish that state before making ordinary services available. ServiceRegistry remains the discovery anchor; this does not make Authentication, ConsumerAuthorization or transport protection optional bootstrap additions.

1. **Provision the protected foundation.** Create the private database schemas/users and install per-system certificates/private keys, trust stores, revocation material and bootstrap identity credentials. Configure `server.ssl.enabled=true` and mutual TLS (`server.ssl.client-auth=need`); publish protected HTTPS/MQTTS service bindings. The ServiceRegistry bootstrap address is configured through `service.registry.address/port`. An address is not a trust anchor.

2. **Start protected discovery.** ServiceRegistry registers its own services in its database and exposes only the scoped bootstrap operations needed by the other Core systems. If a configured filter adds a support dependency, such as Blacklist, that dependency is also a readiness prerequisite. No anonymous or plaintext registry interval is required.
3. **Bring AA to readiness.** Start Authentication and ConsumerAuthorization with the provisioned identities, operator credentials and scoped initial policies. In the reference `OUTSOURCED` profile, a system uses `authenticator.credentials` to obtain an identity token over mutual TLS; verification binds that token to its certificate identity. A narrowly scoped lookup of *identity* may precede possession of the token, but still requires authenticated, encrypted transport. Initial operator/certificate access and explicit bootstrap grants resolve dependency cycles without disabling AA.
4. **Start authorised service selection.** Protect the selected orchestrator and ensure discovery, Authentication and ConsumerAuthorization are ready. DSO uses `enable.authorization=true`; provision SimpleStore rules where that strategy is used. Those rules do not themselves grant service access. DSO's own authorisation/blacklist bootstrap lookups are scoped exceptions to recursive orchestration, not exceptions to the security baseline.
5. **Release Support and application traffic.** Systems register their protected service offers and consume only permitted operations. Providers enforce the verified identity and policy/token decision even after orchestration. Missing trust, an invalid credential or an unavailable required AA decision prevents readiness or denies the request; startup does not fall back to `DECLARED`, plaintext transport or unauthorised service use.

This sequence describes security and dependency readiness, not a strict process ordering for every runtime. Registration, identity and policy bootstrap can require controlled retries under the same security constraints. Unencrypted HTTP/MQTT, self-declared identities and authorisation bypass are explicit isolated-test exceptions; they are not the v5.0 default and do not demonstrate a conforming local-cloud startup.

2.5 Deployed Java Reference Cloud (24 October 2026)

The local Java reference deployment is the Docker Compose project `arrowhead-v52-test-cloud`, maintained in `Arrowhead-test-cloud-java`. Figure 5 records its 22 deployed microsystems, including both orchestration variants, the operations manager and the demonstration application [18]. This is a dated deployment example of the architecture, not a requirement that every local cloud deploy every optional support system. In this 24 October snapshot, the five Core systems and fifteen support systems are Java implementations; **LocalCloudManager** has a Node.js API and **RandomDataProducer** is a Python application. Thus "Java cloud" identifies the reference implementation baseline, not the language of every process.

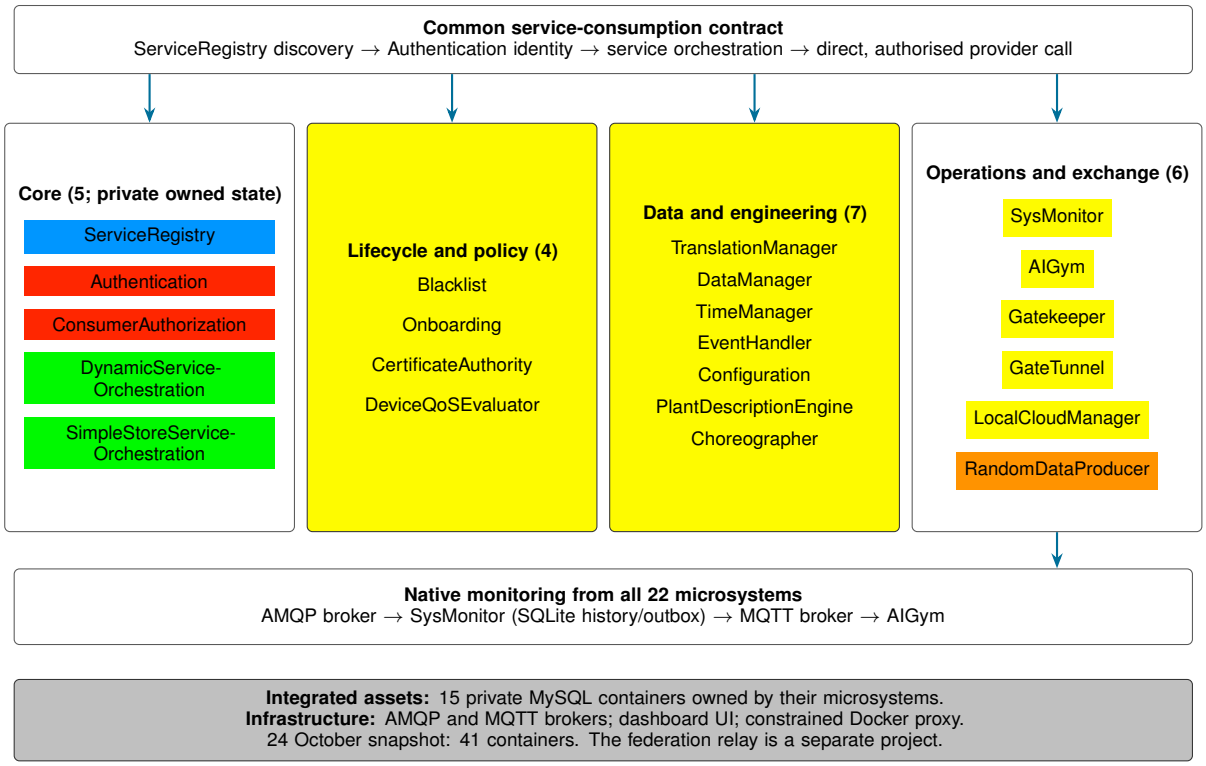


Figure 5: Deployed Java reference cloud. All 22 named systems are microsystems; message brokers, dashboard and Docker proxy are infrastructure; database containers are integrated assets inside their host microsystems. The shared consumption path is drawn once to avoid repeating bootstrap edges.

The 41 running containers comprise the 22 microsystems, 15 dedicated MySQL containers, two message brokers, a dashboard UI and a Docker proxy. The four **CertificateAuthority**, **Gatekeeper**, **GateTunnel** and **SysMonitor** services originally used the combined Java 21 runtime in this dated snapshot. The primary distribution now packages each as an independent Java 21 project with its own SQLite state; this source change is not a blanket live-image rollout. The former delegated **SysMonitorPublisher** is not a deployed microsystem: each system publishes its own *sysMonitor* observations.

Discovery, orchestration and direct consumption. Systems bootstrap through **ServiceRegistry**, locate *identity* and *serviceOrchestration*, obtain a certificate-bound identity token and request an authorised provider for the required operation. The resulting call goes from consumer to provider; the orchestrator does not proxy the application payload. DSO directly discovers its own authorisation and blacklist dependencies to avoid recursive orchestration. Authentication also bootstraps its authorisation check. Other dependency calls use orchestration, with the documented caches and exceptions.

Monitoring and application data. Native publishers send observations over AMQPS to RabbitMQ; **SysMonitor** consumes authorised selections and stores them in its own SQLite history/outbox. It republishes

normalised *sysMonitorData* through MQTT for **AIGym**. **LocalCloudManager** reads *sysMonitorHistory* and manages subscriptions. The demonstration producer instead writes application SenML data to **DataManager** every 30 s through *dataHistorian* `put-data`. Monitoring history is not stored by DataManager.

Engineering, workflow and federation paths. The plant editor calls **PlantDescriptionEngine**; an active plant is compiled into an atomic DSO rule snapshot through *serviceOrchestrationPlantManagement*. **Choreographer** asks DSO for push orchestration on behalf of workflow executors; no executor is deployed by default. Onboarding consumes certificate signing, validation and revocation, identity management and blacklist operations. Inter-cloud selection uses Gatekeeper discovery/negotiation and an AH-GT-1 GateTunnel session over the external relay. The 24 October deployment snapshot contained neither SimpleStore route rows nor an active DSO plant-rule row: these mechanisms are available, but the diagram does not imply an active engineered workflow.

Reference security profile. The deployed service profile realises the v5.0-and-later default: full AA and information encryption, using mutual TLS with `CERT_AUTH`, certificate-bound identity tokens and operation-scoped authorisation. Unsecured component samples are explicit development exceptions, not an alternative architectural default. The operator console requires login and reaches Docker only through a project-scoped proxy. CertificateAuthority publishes CRLs; services and brokers enforce them on new TLS handshakes. Ending established connections is a separate action. AH-GT-1 protects relayed payloads end to end; cloud application trust remains separate from relay/organisation trust [18, 19].

2.6 Core Control Flow in the Deployed Cloud

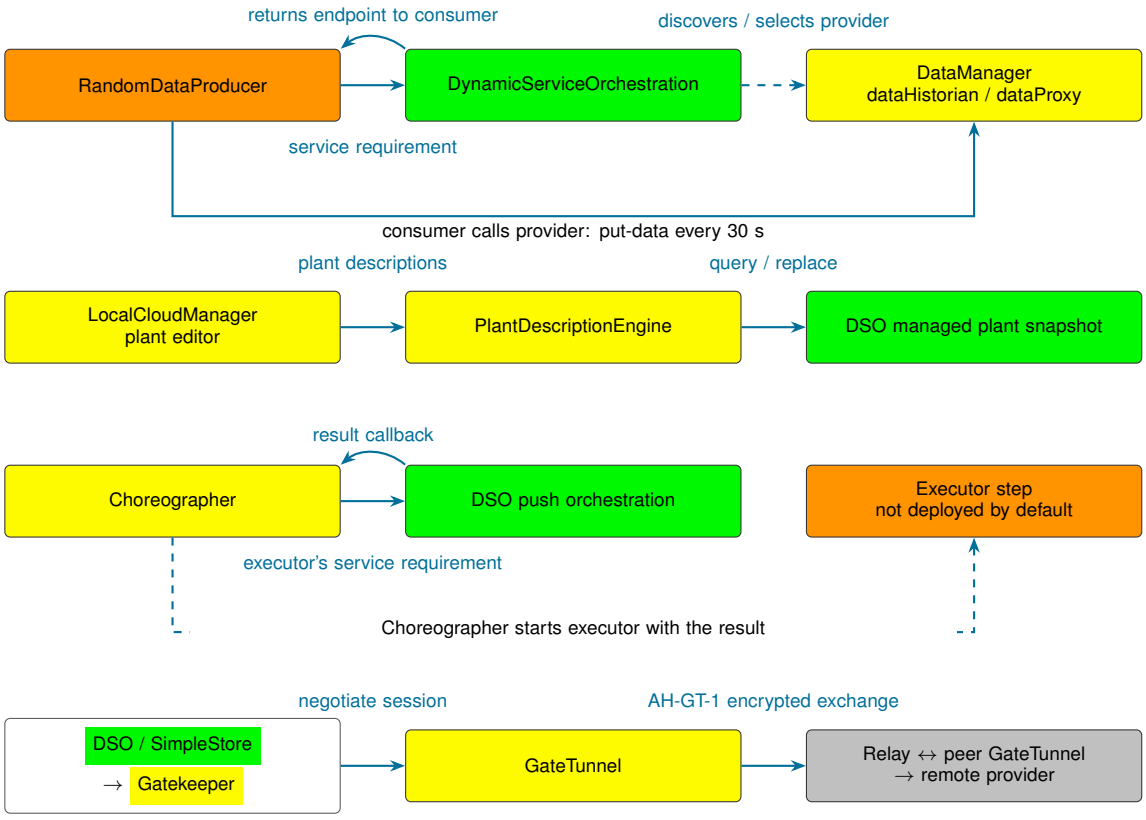


Figure 6: Principal application, plant, workflow and federation paths in the Java reference deployment. The first row separates provider selection from the direct application call. Dashed executor use is an available capability, not a deployed executor.

The implementation uses `http.collector.mode=ORCHESTRATION` for ordinary Spring dependency

calls. Bootstrap discovery of the registry, identity and orchestrator services is exempt; DSO also bootstraps ConsumerAuthorization and Blacklist, while Authentication bootstraps its own authorisation dependency. Provider results are subject to operation-specific policy grants. A configured authorisation grant is permission to consume a service, not evidence that a request is currently executing. SimpleStore and DSO keep independent databases; the active plant snapshot belongs to DSO and does not require a SimpleStore rule.

3 Microsystem White-Box Descriptions

3.1 ServiceRegistry

Purpose. Holds the authoritative registry of *devices*, *systems*, *service definitions*, *service instances* and *interface templates* of the local cloud. Compared with the abstract description in [2], 5.2.2 adds devices and systems as first-class registry objects, so that physical deployment can be expressed.

Information model. A *device* (name, addresses of type `MAC`, `IPV4`, `IPV6`, `HOSTNAME`, metadata) hosts one or more *systems* (name, addresses, version, metadata). A system provides *service instances*; an instance is identified by `<SystemName>|<serviceDefinition>|<version>`, has an expiry time, metadata and one or more *interfaces*. Each interface instantiates an *interface template* whose *template properties* (with mandatory flags and optional validators) constrain the interface properties, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations` for `generic_http(s)`. Each interface also carries a *policy* (`NONE`, `CERT_AUTH`, `TIME_LIMITED_TOKEN_AUTH`, `USAGE_LIMITED_TOKEN_AUTH`, `BASE64_SELF_CONTAINED_TOKEN_AUTH`, `RSA_SHA256/512_JSON_WEB_TOKEN_AUTH`, `TRANSLATION_BRIDGE_TOKEN_AUTH`) which tells the orchestration which kind of token a consumer needs.

Produced services. *deviceDiscovery*, *systemDiscovery* and *serviceDiscovery* (each `register`, `lookup`, `revoke`; flagged `unrestrictedDiscovery`), *serviceRegistryManagement* (bulk CRUD on devices, systems, service definitions, service instances and interface templates) and *generalManagement*.

Key behaviour.

- **Registration ownership**: a system may only register/revoke itself and its own service instances (the requester's authenticated name must match).
- **Discovery policy**: with `restricted` (default), a `lookup` by an ordinary system returns only instances whose metadata carries `unrestrictedDiscovery=true` (i.e. the public Core services). Systems in `service.discovery.direct.access` (by default **DynamicServiceOrchestration**, **Translation Manager**, **DeviceQoSEvaluator**) may look up everything. This forces application consumers to use orchestration, which applies authorisation.
- **Interface policy**: `restricted` rejects unknown interface templates; `extendable` creates a template with all supplied properties mandatory; `open` creates an unconstrained template.
- **Address policy**: `allow.self.addressing` and `allow.non.routable.addressing` decide whether loopback/link-local and private addresses are accepted.
- **Verbose lookup**: with `discovery.verbose=true`, a lookup may embed provider system and device details in the response.

3.2 Authentication

Purpose. Issues and validates system identities when certificates are not used (`OUTSOURCED` policy in the other systems). It realises the *Authentication system* of [2] for dynamic, token-based identities.

Produced services. *identity* (`identity-login`, `identity-logout`, `identity-change-credentials`, `identity-verify`; unrestricted discovery), *identityManagement* (`identity-mgmt-create`, `identity-mgmt-update`, `identity-mgmt-remove`, `identity-mgmt-query`, `identity-mgmt-session-close`, `identity-mgmt-session-query`) and *generalManagement*.

Key behaviour.

- Identities (system name, sysop flag, authentication method `PASSWORD` and hashed credentials) are created by the operator through *identityManagement*.
- `identity-login` validates the credentials and opens an *ActiveSession* with a random identity token valid for `identity.token.duration` seconds (default 300). The client presents the token as `BearerIDENTITY-TOKEN//<token>`.
- `identity-verify` returns whether a token is valid and, if so, the system name and sysop flag – used by the `OutsourcedFilter` of every other system.
- **Authentication** runs with the fixed `INTERNAL` policy. Its own outbound calls use an *authenticator key*, an HMAC of its system name with `authentication.secret.key`, which the other systems hold in `authenticator.secret.keys`.
- A cleaner job removes expired sessions every `cleaner.job.interval` ms.

3.3 ConsumerAuthorization

Purpose. Realises the *Authorisation system*: stores access-control policies, answers access checks and issues and verifies access tokens.

Produced services.

- *authorization* (`grant`, `revoke`, `lookup`, `verify`; unrestricted): providers grant/revoke *provider-level* policies for their own services; anyone may verify.
- *authorizationToken* (`generate`, `verify`, `get-public-key`, `register-encryption-key`, `unregister-encryption-key`; unrestricted).
- *authorizationManagement* (`grant-policies`, `revoke-policies`, `query-policies`, `check-policies`): *management-level* policies.
- *authorizationTokenManagement* (`generate-tokens`, `query-tokens`, `revoke-tokens`, `add-encryption-keys`, `remove-encryption-keys`).
- *generalManagement*.

Policy engine. A policy header identifies *(cloud, provider, target type, target)*, where the target type is `SERVICE_DEF` or `EVENT_TYPE`; it holds a default policy and optional per-operation (scope) policies. A policy is `ALL`, `WHITELIST` (list of consumers), `BLACKLIST` (list of consumers) or `SYS_METADATA` (a metadata requirement evaluated against the consumer's metadata, fetched from the **ServiceRegistry** through *system Discovery* `lookup`). Evaluation (Figure 7) implements the GSoSD priority rule: management-level policies, set by the operator, override provider-level policies set by the application provider; if neither level has a relevant policy, access is denied.

Token engine. Tokens are bound to (consumer, provider, target, scope):

- **time-limited** (expires after `token.time.limit`, default 60 s) and **usage-limited** (`simple.token.usage.limit` uses, default 5) simple tokens of `simple.token.byte.size` bytes;
- **self-contained** tokens, Base64 or JWT signed with the system's RSA key (SHA-256/512), verifiable offline using `get-public-key`;
- **translation-bridge** tokens, used only by **TranslationManager**.

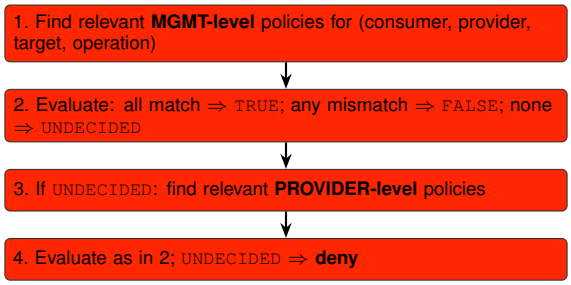


Figure 7: Access decision in `AuthorizationPolicyEngine`.

A provider can register an encryption key, after which tokens issued for its services are returned encrypted for it. Token secrets and provider keys are stored encrypted with `secret.cryptographer.key`. Systems listed in `unbounded.token.generation.whitelist` (by default **DynamicServiceOrchestration** and **Translation Manager**) may generate tokens on behalf of other consumers. Tokens are deleted after `token.max.age` minutes.

3.4 DynamicServiceOrchestration

Purpose. Realises the *advanced* orchestration approach of [2]: it computes at request time which provider instances a consumer should use and returns everything needed to consume them (addresses, interfaces, tokens).

Produced services. *serviceOrchestration* (`pull`, `subscribe`, `unsubscribe`; unrestricted discovery), *serviceOrchestrationPushManagement* (`subscribe`, `unsubscribe`, `trigger`, `query`), *serviceOrchestrationLockManagement* (`create`, `query`, `remove`), *serviceOrchestrationHistoryManagement* (`query`) and *generalManagement*. The system and its *serviceOrchestration* service carry the metadata key `orchestrationStrategy`.

Orchestration request. An *orchestration form* names the target service definition and optionally operations, versions, interface templates and properties, security policies, metadata requirements, preferred providers, QoS requirements, an exclusivity duration and orchestration *flags*: `MATCHMAKING`, `ONLY_PREFERRED`, `ONLY_EXCLUSIVE`, `ALLOW_TRANSLATION`, `ALLOW_INTERCLOUD`, `ONLY_INTERCLOUD`. Each pull or push creates an *OrchestrationJob* record (the orchestration history).

Local orchestration pipeline. Figure 8 shows the candidate pipeline implemented in `LocalServiceOrchestration`. Each step can only remove candidates; an empty set terminates the job with an empty result and warnings.

Push orchestration. Consumers (or operators, via push management) create *subscriptions* with a notify interface (`HTTP`, `HTTPS`, `MQTT`, `MQTTS`) and optional duration. A `trigger` (or an immediate trigger at subscription time) queues jobs that a bounded worker pool executes, delivering results to the subscriber. **Locks** reserve exclusive instances until their expiry and can be managed by operators. A cleaner job removes expired subscriptions and locks and history older than `orchestration.history.max.age` days.

3.5 SimpleStoreServiceOrchestration

Purpose. Realises the *atomic* orchestration approach of [2]: an operator-maintained table maps consumers to concrete service instances.



Figure 8: Candidate pipeline of **DynamicServiceOrchestration** (local cloud).

Produced services. *serviceOrchestration* (`pull`, `subscribe`, `unsubscribe`), *serviceOrchestration SimpleStoreManagement* (`create`, `query`, `remove`, `modify-priorities`), *serviceOrchestrationPush Management*, *serviceOrchestrationHistoryManagement* and *generalManagement*.

Key behaviour. Each *OrchestrationStore* rule is (consumer, service definition, service instance identifier, priority). A pull returns the consumer's rules for the requested service definition ordered by priority, re-ordered by preferred providers (`ONLY_PREFERRED` drops others), and reduced to the highest-priority rule per service definition with `MATCHMAKING`. The response contains only the instance identifier, provider name, service definition and version; the consumer resolves access details itself (from the **ServiceRegistry** or local configuration). The system consumes no other Core services during orchestration; consequently it does not check authorisation or issue tokens, and keeping rules consistent with the registry is the operator's task – exactly the trade-off identified in [2].

3.6 Interacting Support Systems (5.2.2)

Support system	Port	Produced services	Used by the Core
Blacklist	8464	*blacklistDiscovery*, *blacklistManagement*, *generalManagement*	Blacklist filter of every Core system (`check`); **Dynamic ServiceOrchestration** candidate filtering (`query`).
TranslationManager	8465	*translationBridge*, *translationBridge Management*, *translationReport*, *general Management*	**DynamicServiceOrchestration** interface/data-model translation (`discovery`, `negotiation`).
DeviceQoSEvaluator	8472	*qualityEvaluation*, *deviceQualityData Management*, *generalManagement*	**DynamicServiceOrchestration** QoS filtering/sorting.

Table 5: 5.2.2 Support systems the Core interacts with.

3.7 Plant-Managed Dynamic Orchestration (Local Extension)

The Java implementation dated 23 October 2026 changes the PDE backend from SimpleStore to **Dynamic ServiceOrchestration**. PDE atomically publishes the complete active-plant snapshot using *serviceOrchestration PlantManagement*: `query` is `POST/serviceorchestration/mgmt/plant-rules/query`; `replace` is `PUT/serviceorchestration/mgmt/plant-rules/replace`. This is a local Java extension; it is not part of the unmodified upstream 5.2.2 contract.

A scope is a consumer and service definition with ranked service-instance identifiers. Lower nonnegative priority wins among eligible providers. Empty provider lists retain managed scopes with no result; an empty scope list clears the active plant. DSO first restricts candidates to the engineered set, then retains its normal authorisation, blacklist, lock, interface and QoS checks. With matchmaking, it selects the best surviving plant priority; without matchmaking it returns the survivors in priority order. Consumer preferences can narrow this set, but cannot introduce an unengineered provider. Managed scopes cannot escape through local failure, `ALLOW_INTERCLOUD` or `ONLY_INTERCLOUD`.

The `orchestration_plant_rules` table stores one atomic snapshot in DSO's database, retaining it across restarts. Replacement requires both management-policy permission and an authenticated PDE or Sysop identity; LocalCloudManager may query only. PDE retains its own plant-description and alarm stores. An unresolved producer produces an empty managed scope, rather than deleting the consumer's engineering constraint. No active description clears the snapshot. Query/replacement failures leave PDE degraded rather than claiming that the plant is in force.

Apply `database_patch_pde_dso.sql` to an existing DSO database before starting the upgraded runtime. The Java test-cloud migration target creates the table and grants data access, and configures operation-scoped query/replace permissions. Verify PDE's successful synchronisation before relying on the new constraints. Existing consumer orchestration caches (normally 300 s) can delay visible changes beyond the PDE's 15 s interval. Legacy PDE-owned SSO entries are not automatically deleted; optional cleanup must preserve operator-owned SSO rules. SimpleStore remains an independent optional backend, but PDE no longer requires it. This update records source/configuration changes, not a live deployment or cross-language conformance result.

4 Services

All services are versioned `1.0.0` and are offered over `generic_http` or `generic_https` (depending on `server.ssl.enabled`) and, when enabled, over `generic_mqtt`/`generic_mqtts`. The service-specific SD and IDD documents are published in the implementation documentation [15].

4.1 Produced Services

Microsystem	Service definition	Operations	HTTP base path
ServiceRegistry	*deviceDiscovery*†	register, lookup, revoke	`/serviceregistry/device-discovery`
	systemDiscovery†	register, lookup, revoke	`/serviceregistry/system-discovery`
	serviceDiscovery†	register, lookup, revoke	`/serviceregistry/service-discovery`
	serviceRegistryManagement	device-, system-, service-definition-, service-, interface-template-query/create/(update)/remove	`/serviceregistry/mgmt`
	generalManagement	get-log, get-config	`/serviceregistry/general/mgmt`
Authentication	*identity*†	identity-login, identity-logout, identity-change-credentials, identity-verify	`/authentication/identity`
	identityManagement	identity-mgmt-create/-update/-remove/-query, identity-mgmt-session-close/-query	`/authentication/mgmt`

Microsystem	Service definition	Operations	HTTP base path
	generalManagement	get-log, get-config	`/authentication/general/mgmt`
Consumer Authorization	*authorization*†	grant, revoke, lookup, verify	`/consumerauthorization/authorization`
	authorizationToken†	generate, verify, get-public-key, register-encryption-key, unregister-encryption-key	`/consumerauthorization/authorization-token`
	authorizationManagement	grant-policies, revoke-policies, query-policies, check-policies	`/consumerauthorization/authorization/mgmt`
	authorizationTokenManagement	generate-tokens, query-tokens, revoke-tokens, add-encryption-keys, remove-encryption-keys	`.../authorization/mgmt/token`
	generalManagement	get-log, get-config	`/consumerauthorization/general/mgmt`
DynamicService-Orchestration	*serviceOrchestration*†	pull, subscribe, unsubscribe	`/serviceorchestration/orchestration`
	serviceOrchestration PushManagement	subscribe, unsubscribe, trigger, query	`.../mgmt/push`
	serviceOrchestration LockManagement	create, query, remove	`.../mgmt/lock`
	serviceOrchestration HistoryManagement	query	`.../mgmt/history`
	generalManagement	get-log, get-config	`/serviceorchestration/general/mgmt`
	serviceOrchestrationPlant Management	query, replace	`/serviceorchestration/mgmt/plant-rules`
SimpleStoreService-Orchestration	*serviceOrchestration*†	pull, subscribe, unsubscribe	`/serviceorchestration/orchestration`
	serviceOrchestrationSimpleStore Management	create, query, remove, modify-priorities	`.../mgmt/simple-store`
	serviceOrchestration PushManagement	subscribe, unsubscribe, trigger, query	`.../mgmt/push`
	serviceOrchestration HistoryManagement	query	`.../mgmt/history`
	generalManagement	get-log, get-config	`/serviceorchestration/general/mgmt`

Table 6: Services produced by the 5.2.2 Core microsystems. † = registered with `unrestrictedDiscovery=true` (directly discoverable in `restricted` mode).

4.2 Consumed Services

Consumer	Service & operation (provider)	When
All except **ServiceRegistry**	*systemDiscovery*, *serviceDiscovery*: register, revoke (**ServiceRegistry**)	Start-up and shutdown (`ApplicationInitListener`).
All with `OUTSOURCED` policy	*identity*: identity-login, identity-logout (**Authentication**)	Start-up/shutdown, to obtain own identity token.
All with `OUTSOURCED` policy	*identity*: identity-verify (**Authentication**)	Each inbound request (OutsourcedFilter).
All with `AUTHORIZATION` management policy	*authorization*: verify (**Consumer Authorization**)	Each inbound `/mgmt/` request from a non-sysop, non-whitelisted system.
All with blacklist filter	*blacklistDiscovery*: check (**Blacklist**)	Each inbound request.
ConsumerAuthorization	*systemDiscovery*: lookup (**Service Registry**)	Evaluating `SYS_METADATA` policies.
DynamicService Orchestration	*serviceDiscovery*: lookup (**Service Registry**)	Every orchestration (step 1).

Consumer	Service & operation (provider)	When
DynamicService Orchestration	*authorizationManagement*: check-policies; *authorizationToken Management*: generate-tokens (**ConsumerAuthorization**)	When `enable.authorization=true`.
DynamicService Orchestration	*blacklistManagement*: query (**Blacklist**)	When the blacklist filter is enabled.
DynamicService Orchestration	*translationBridgeManagement*: discovery, negotiation (**Translation Manager**)	When translation is enabled and allowed.
DynamicService Orchestration	*qualityEvaluation* (**DeviceQo SEvaluator**)	When QoS requirements are given and `enable.qos=true`.
SimpleStoreService Orchestration	–	No Core services consumed during orchestration.

Table 7: Services consumed by the 5.2.2 Core microsystems.

5 Use Cases (White Box)

The use cases of [2] are revisited here with the concrete 5.2.2 services, operations and filters. Numbered circles refer to the explanation below each diagram.

5.1 Bootstrapping and Service Registration

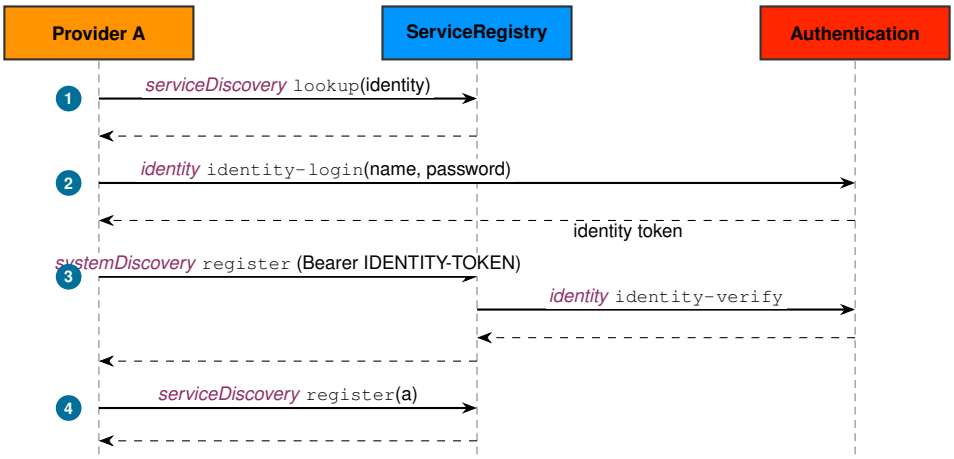


Figure 9: Registration with certificate-bound `OUTSOURCED` authentication on mutually authenticated, encrypted channels. TLS handshakes are omitted.

① A looks up *identity*; this exact bootstrap lookup does not require an identity token yet. It still requires mutual TLS and a trusted caller certificate.

② A logs in and receives an identity token (default 300 s).

③ A registers itself; the **ServiceRegistry** verifies the token with **Authentication** before accepting.

④ A registers its service *a* with its interfaces and interface policy. With `CERTIFICATE` policy steps 1–2 and the verification are replaced by certificate-derived identity on mutual TLS. The isolated `DECLARED` test mode does not satisfy this secure use case. Provider/service authorisation remains required independently of the chosen identity mechanism.

5.2 Dynamic Orchestration with Authorisation and Tokens

① B (which may only discover the orchestration service directly in `restricted` mode) sends an orchestration form.

② The orchestrator, having direct access, looks up all matching instances and applies locks, exclusivity and preferences (Figure 8).

③ Candidates for which B is not authorised (management level overriding provider level) are removed.

④ For the remaining candidates whose interface policy requires a token, the orchestrator generates tokens on B's behalf (it is in the `unbounded.token.generation.whitelist`). The response contains the full instance description and the token(s).

⑤ B consumes *a*. A verifies the token – online via *authorizationToken* `verify` for simple tokens, or offline using the public key for self-contained/JWT tokens. A may alternatively check access with *authorization* `verify`.

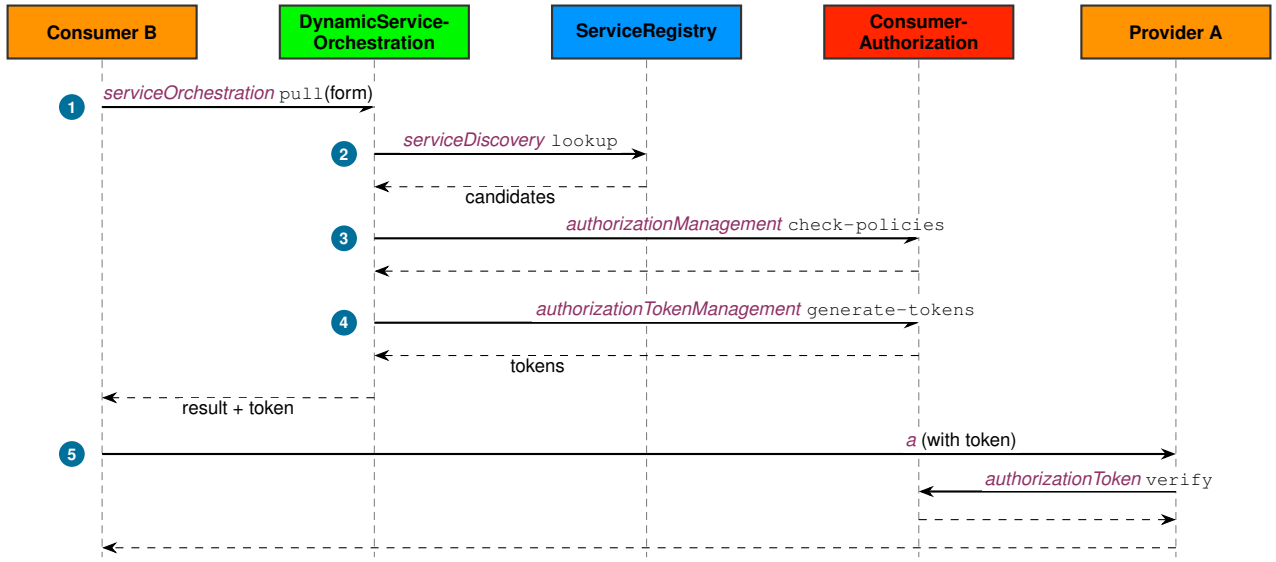


Figure 10: Advanced (dynamic) orchestration, pull mode, with authorisation enabled.

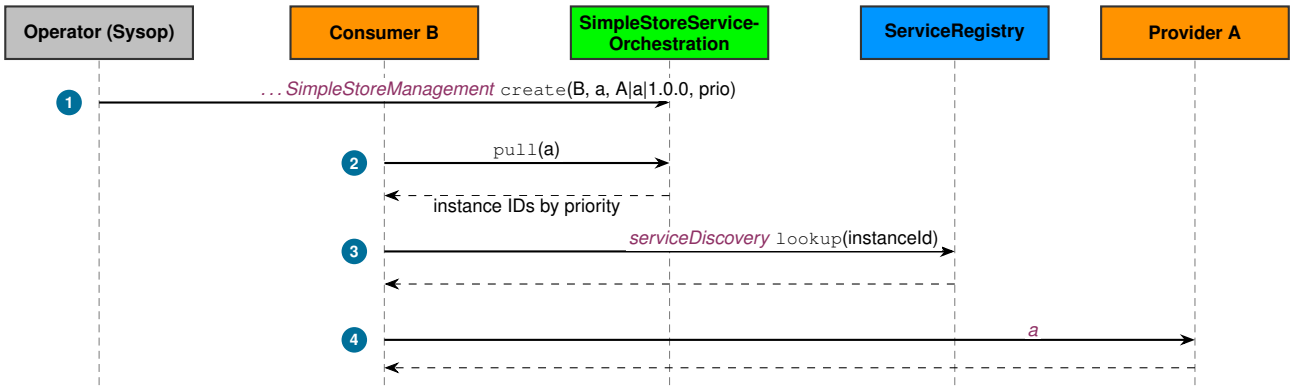


Figure 11: Atomic orchestration with the prioritised rule store.

5.3 Atomic (Simple Store) Orchestration

① The operator provisions rules. ② B pulls and receives only identifiers. ③ B resolves access details (requires discovery access in the **ServiceRegistry**, i.e. `open` policy or B in `service.discovery.direct.access`). ④ B consumes *a*; any access check is up to A.

5.4 Protected Management Access

When an application system (e.g. a plant-description tool) calls a management operation such as *service RegistryManagement* `service-query`, the management filter of the **ServiceRegistry** maps the path and method to (service definition, operation) and asks **ConsumerAuthorization** *authorization* `verify`. Access therefore requires a management-level policy for the caller on the target system's management service, unless the caller is the operator or listed in `management.whitelist`.

6 Control and Data Planes

All Core services belong to the control plane. In 5.2.2 the separation is concretely visible:

- With **DynamicServiceOrchestration** and self-contained (JWT) tokens, a consumer and provider can verify the access token locally while it remains valid. Independent identity, policy and revocation requirements still apply; with simple tokens, each token verification is a control-plane call.
- The inbound filter chain (Section 2.3) introduces control-plane calls into *every* request only for the `OUTSOURCED` policy, the blacklist filter and management requests. Certificate-based authentication keeps request authentication local to the TLS handshake.
- **SimpleStoreServiceOrchestration** reduces repeated provider-selection work. Its stored rules and cached endpoint lookups do not remove the AA and encryption baseline. Required identity, authorisation and revocation dependencies must remain available; a missing required check denies access rather than producing an unsecured fallback.

7 Security

7.1 Default Secure Profile and Development Exceptions

Since v5.0, full AA security and encryption of exchanged information are the default. In the reference profile, HTTPS/MQTTS and mutual TLS protect peer identity, confidentiality and integrity; the system certificate must comply with the Arrowhead profile and chain to the configured local-cloud trust anchor. The `OUTSOURCED` identity token is verified and bound to that certificate, and ConsumerAuthorization policies or valid scoped access tokens govern every service operation. Management and bootstrap exchanges obey the same baseline.

The implementation also contains development switches, including `server.ssl.enabled=false` and `authentication.policy=DECLARED`. Their existence, or their appearance in a sample `application.properties`, does not redefine the architectural default. They must be explicitly selected for an isolated test and are non-conforming to the normal cloud profile. A runtime must not silently select them when TLS, credentials or required AA services fail. Enabling TLS alone is not sufficient: authorisation enforcement must also be active.

7.2 Security Model

- **Protocols**: encrypted HTTPS REST with JSON, and MQTTS/AMQPS where broker interfaces are used. Protection covers discovery, identity and policy traffic, management, monitoring and application information. An unsecured HTTP/MQTT transport is not the default service binding.
- **System authentication**: `CERTIFICATE` (X.509, CN-derived name, same local cloud, profile types `SYSTEM` or `OPERATOR`); `OUTSOURCED` (identity token from **Authentication**, or HMAC authenticator key); `DECLARED` (no authentication – an explicit isolated-test exception, never the default secure profile). The certificate profile types `MASTER`, `GATE`, `ORGANIZATION`, `LOCAL_CLOUD`, `ON_BOARDING`, `DEVICE`, `SYSTEM`, `OPERATOR` are recognised.
- **Operator (sysop)**: identified by an `OPERATOR` certificate, a sysop identity in **Authentication**. Trust in the declared name `Sysop` belongs only to the non-conforming development mode.
- **Produced-service authorisation**: management endpoints are protected centrally by the management filter; application providers use **ConsumerAuthorization** and/or tokens. Management-level policies take precedence over provider-level ones; default is deny. Orchestration-time eligibility does not replace this provider-side enforcement.
- **Discovery restriction**: `restricted` discovery policy prevents consumers from bypassing orchestration-time authorisation.
- **Data protection**: token secrets and provider encryption keys are encrypted at rest (16-byte `secret.cryptographer.key`); passwords are stored hashed; tokens can be delivered encrypted with the provider's registered key. These token/key protections complement transport encryption; they do not imply that every application database is encrypted at rest. The storage protection needed for each information class is

specified separately. Broker TLS protects each hop; the selected federation profile additionally protects application bodies end to end across the relay.

- **Blacklisting**: optional cloud-wide denial of named systems on every request and during orchestration; `force.blacklist.filter` makes the check fail closed.
- **Credential provisioning and admission**: the Core verifies credentials but does not decide who receives them. The **Onboarding** Support system adds that decision as a further security mechanism: only operator-approved devices and systems obtain certificates and **Authentication** identities, and off-boarding withdraws registry entries, identities and certificates again. For this, **Onboarding** is a management client of the **ServiceRegistry** (device and system entries) and of **Authentication** (identities) [1, 17].
- **Management layer**: the **PlantDescriptionEngine** is a management client of the **ServiceRegistry** (registry queries) and of **DynamicServiceOrchestration** (managed active-plant scopes and provider priorities). It enforces engineered plant descriptions through these Core management services only, so the management policies of the Core systems also govern the management layer [1, 17].

8 Non-Functional Requirements as Realised

Accurate enough message caching. The `ServiceCollector` caches resolved Core service models; identity tokens and self-contained tokens allow verification results to be reused for their validity period. Expiry-based caches (short token lifetimes, 60 s temporary locks) keep the risk of stale data bounded.

Relative system independence. Each system owns its schema; only the **ServiceRegistry** is mandatory; all other interactions are feature-flagged and fail with explicit warnings or, where configured, fail closed.

Backwards compatibility. 5.x is intentionally incompatible with 4.x. Within 5.x, the service definitions and operations listed in Section 4 are versioned `1.0.0`; 5.2.x adds database patch scripts (`database_patch_520.sql`) for in-place upgrade.

Naming convention. Enforced by validators and the `extended` normalisation mode (PascalCase systems, camelCase services, kebab-case operations, snake_case interface templates) [16].

Active vs. passive systems. Pull orchestration serves active systems; push orchestration (subscriptions with notify interfaces) serves passive systems that wait for stimuli.

9 Port Numbering Specification

Arrowhead systems discover each other through the **ServiceRegistry**, so in principle any port works. In practice fixed, well-known default ports matter: the **ServiceRegistry** port must be known for bootstrapping, test and reference deployments put many systems on one host, firewall rules and container mappings are written against them, and several local clouds are often run side by side. Up to 5.2.2 the defaults were assigned per repository without a common plan, which led to the same default port (8465) being used by **SimpleStore ServiceOrchestration**, **TranslationManager** and **Onboarding**. This section specifies how ports are allocated within a local cloud and across a system of local clouds. The key words *shall*, *should* and *may* are used as in RFC 2119.

9.1 Principles

P1 – One canonical port per system type. Every Arrowhead framework system type (Core and Support) *shall* have exactly one canonical default port, unique across the whole framework allocation (Table 9). Any combination of framework systems can thereby be co-located on one host without reconfiguration.

P2 – One port per system instance. A system *shall* serve all its HTTP(S) services on its canonical port; the same port is used in secure and insecure mode. Additional listeners (health or sidecar endpoints) *shall* use the health range.

P3 – Stability. Allocated ports are never changed without a Roadmap WG decision. Ports of retired systems stay *reserved* for at least one major version and are not re-assigned. The Arrowhead 4.6 allocations are kept wherever they are unique.

P4 – Discovery over configuration. Only the **ServiceRegistry** port is needed for bootstrapping. Consumers *shall not* hard-code any other port; the registered `accessPort` of a service interface is authoritative.

P5 – Reachability of registered ports. The port a system registers *shall* be the port its consumers use. If address or port translation is applied (NAT, container port publishing), the registered value *shall* be valid for the consumers of the local cloud, or translation *shall* follow the cloud offset rule of Section 9.4.

P6 – Ephemeral-range avoidance. Allocated ports *shall* lie below 32768, the start of the Linux ephemeral port range, so that outgoing connections never occupy them.

9.2 Port Block of a Local Cloud

A local cloud uses the 1000-port *block* 8000–8999 (cloud index $k = 0$), divided into functional ranges (Table 8).

Databases and other private back ends of a system are not part of the block: they *should* not be published outside the host or container network. If they must be, the operator chooses ports outside the block.

Range	Use	Allocation authority
8000–8099	Tunnel control port pool (**GateTunnel** session control ports)	Allocated at run time by **GateTunnel**
8100–8199	Reserved for future framework ranges	Roadmap WG
8200–8299	Inter-cloud session listeners (**GateTunnel**, one per negotiated consumer)	Allocated at run time by **GateTunnel**
8300–8429	Reserved for future framework ranges	Roadmap WG
8430–8439	Life cycle and engineering systems	Roadmap WG
8440–8449	Core systems and trust infrastructure	Roadmap WG
8450–8469	Support systems	Roadmap WG
8470–8479	Monitoring, QoS and analytics systems	Roadmap WG
8480–8489	Health and sidecar endpoints of framework systems	Roadmap WG
8490–8499	Operator tools (e.g. LocalCloudManager)	Roadmap WG
8500–8799	Application systems of the local cloud	Local cloud operator
8800–8899	Messaging infrastructure (8883 MQTT over TLS)	Roadmap WG
(5671)	AMQP broker for *sysMonitor* (well-known port, outside the block)	Roadmap WG
8900–8999	Reserved	Roadmap WG

Table 8: Functional ranges of the port block of a local cloud ($k = 0$).

9.3 Canonical Port Allocation

Port	System	Status	Remark
8432	**PlantDescriptionEngine**	allocated	as in 4.6
8435	**Onboarding**	allocated	4.6 Onboarding controller port; was 8465 in the 5.2.2 migration
8437	(SystemRegistry)	reserved	4.6, replaced by **ServiceRegistry**
8439	(DeviceRegistry)	reserved	4.6, replaced by **ServiceRegistry**
8441	**DynamicServiceOrchestration**	allocated	4.6 Orchestrator port
8442	**SimpleStoreServiceOrchestration**	allocated	was 8465 up to 5.2.2
8443	**ServiceRegistry**	allocated	bootstrap port
8444	**Authentication**	allocated	
8445	**ConsumerAuthorization**	allocated	4.6 Authorization port
8446	–	free (core)	next Core system
8447	(hawkBit configuration manager)	reserved	4.6
8448	**CertificateAuthority**	allocated	as in 4.6
8449	**Gatekeeper**	allocated	as in 4.6
8451	(QoS Monitor)	reserved	4.6, replaced by **DeviceQoSEvaluator**
8453	**GateTunnel**	allocated	4.6 Gateway port
8455	**EventHandler**	allocated	as in 4.6
8457	**Choreographer**	allocated	as in 4.6
8459	**Configuration**	allocated	as in 4.6
8461	**DataManager**	allocated	as in 4.6
8462	(Translator)	reserved	4.6, replaced by **TranslationManager**
8463	**TimeManager**	allocated	as in 4.6
8464	**Blacklist**	allocated	
8465	**TranslationManager**	allocated	sole owner from 5.2.2 on
8450, 8452, 8454, 8456, 8458, 8460, 8466–8469	–	free (support)	

Port	System	Status	Remark
8472	**DeviceQoSEvaluator**	allocated	
8475	**SysMonitor**	allocated	
8478	**AIGym**	allocated	
8470, 8471, 8473, 8474, 8476, 8477, 8479	–	free (monitoring)	
8480	**Gatekeeper** health	allocated	
8481	**GateTunnel** health	allocated	
8482	**SysMonitor** health	allocated	
8483–8489	–	free (health)	
8490	LocalCloudManager UI	allocated	
8491	LocalCloudManager API	allocated	
8501, 8502	(MSCV, GAMS)	reserved	4.6 application-level systems
8883	MQTT broker (TLS)	allocated	IANA *secure-mqtt*; *sysMonitorData* and other intra-cloud MQTT traffic
5671	AMQP broker (TLS)	allocated (exception)	IANA *amqps*; carries only *sysMonitor*; outside the block, offset applies ($5671 + 1000k$)

Table 9: Canonical port allocation (cloud index $k = 0$).

Plain MQTT (1883) and other unencrypted infrastructure ports are permitted only for development on isolated hosts; they are outside the block and are not subject to the cloud offset.

9.4 Systems of Local Clouds

When several local clouds share hosts or addresses – test beds, edge servers hosting more than one cloud, or a system of local clouds deployed on common infrastructure – their port blocks *shall not* overlap. Each such cloud is given a *cloud index* k and uses the block shifted by $1000 \cdot k$:

$$port(s, k) \;=\; canonical(s) \;+\; 1000 \cdot k, \qquad k \in \{0, 1, \ldots, 23\}.$$

- The upper bound $k \leq 23$ keeps every port below 32768 (P6). The cloud with k uses $8000 + 1000k$ to $8999 + 1000k$, e.g. $k = 1$: 9000–9999, $k = 2$: 10000–10999.
- Clouds on disjoint hosts and addresses *may* all use $k = 0$; the cloud index only has to be unique among clouds that share an address.
- The cloud index is assigned by the operator of the shared infrastructure and recorded with the cloud's identity (cloud name, organisation) in the description of the system of local clouds. For the 5.2.2 test beds: `Arrowhead-test-cloud` $k = 0$, `Arrowhead-test-cloud-go` $k = 1$.
- The offset applies to every range of the block, including the **GateTunnel** control pool and session listeners, health and operator ports, and the MQTT broker (8883 $\rightarrow$ 9883 for $k = 1$). It applies equally to the AMQP broker (5671 $\rightarrow$ 6671 for $k = 1$).
- The *inter-cloud relay* is organisation infrastructure shared by the local clouds and belongs to none of their blocks. It listens on 8883 (MQTT over TLS) on its own address; where it must be published on a host that also carries local clouds, a port outside their blocks is used (the test beds use 8893).
- Two deployment styles are conformant: *(a) offset ports* – the systems listen on $port(s, k)$ and register it; or *(b) canonical inside, offset outside* – the systems listen on and register the canonical port inside an isolated network (e.g. a container network), and only the published host ports carry the offset. Style (b) satisfies P5 only for consumers inside that network; consumers outside it reach the cloud through the offset ports and *shall* be configured with the offset **ServiceRegistry** port.
- Inter-cloud contacts (**Gatekeeper** cloud records, relays) *shall* carry the explicit port of the neighbour cloud's **Gatekeeper** ($8449 + 1000k$); a cloud *shall not* infer it.

9.5 Allocation Procedure

1. A new framework system obtains its canonical port from the free ports of the matching range through the Roadmap WG, which updates Table 9.
2. Its implementation *shall* use that port as the shipped default in all configuration artefacts (application properties, code defaults, container images, compose files and documentation).
3. Application systems obtain ports from 8500–8799 from the local cloud operator; these allocations are local to the cloud.

9.6 Conformance of the 5.2.2 Implementations

Item	Status	Action
SimpleStoreService Orchestration default 8465	fixed in workspace	Changed to 8442 (source default, test clouds). The upstream container image still carries 8465 in its built-in configuration and copies it at every container start; the test clouds therefore set `SERVER_PORT=8442`. Upstream change proposed.
Onboarding default 8465	fixed in workspace	Changed to 8435 (code default, properties, IDD, SysDD, test clouds).
TranslationManager 8465	conformant	Sole owner of 8465.
Test-cloud random-data-producer on container port 8463 (**Time Manager**'s port)	fixed in test clouds	Moved to 8500 (application range, operator allocation); **TimeManager** is now published on its canonical 8463 instead of 8467.
DataManager on 8462 in the test clouds (`SERVER_PORT=8462`)	fixed in test clouds	Runs on and registers the canonical 8461; 8462 stays reserved.
LocalCloudManager UI on 8080, API on 8790	fixed in test clouds	Moved to 8490 (UI) and 8491 (API), clearing the tunnel pool.
Plain MQTT 1883 as `mqtt.broker.port` default in Core configurations	accepted (development)	Production deployments use 8883.
Test cloud $k = 1$ (`Arrowhead-test-cloud-go`)	conformant, style (b)	Host ports carry $+1000$ for every system (e.g. 9443, 9490, 9883); systems register canonical ports inside the container network.
AMQP broker of test cloud $k = 1$ published on 9671	fixed in test clouds	Now published on 6671 ($5671 + 1000k$). Moving the AMQP broker into the messaging range (e.g. 8871) to remove the exception remains a Roadmap WG option.
GateTunnel control pool configured as 8000–8100	fixed in test clouds	Upper bound set to 8099 (compose files and code default).
Inter-cloud relay published on host port 8893	conformant	Relay is outside the cloud blocks.
Test cloud $k = 2$ (`Arrowhead-test-cloud-rust`) on host ports 85xx with container ports 8465 (**SimpleStoreService Orchestration**), 8476 (**Onboarding**) and 8480 (LocalCloudManager)	fixed in test clouds	Systems listen on canonical ports (8442, 8435, 8490) and are published on canonical $+2000$ (e.g. 10443, UI 10490, API 10491).

Table 10: Conformance of the 5.2.2 implementations and test beds with the port specification.

10 Interoperability with the Java Reference Implementation

Interoperability is assessed at an explicit service-contract boundary, independently of implementation language. Three different claims must be distinguished: *core-slot substitution* within one local cloud, *federated service consumption* between independently trusted clouds, and *mixed-language providers* registered in the same local cloud. Passing one boundary does not prove complete replacement of every Java API, persistent state format or operational behaviour.

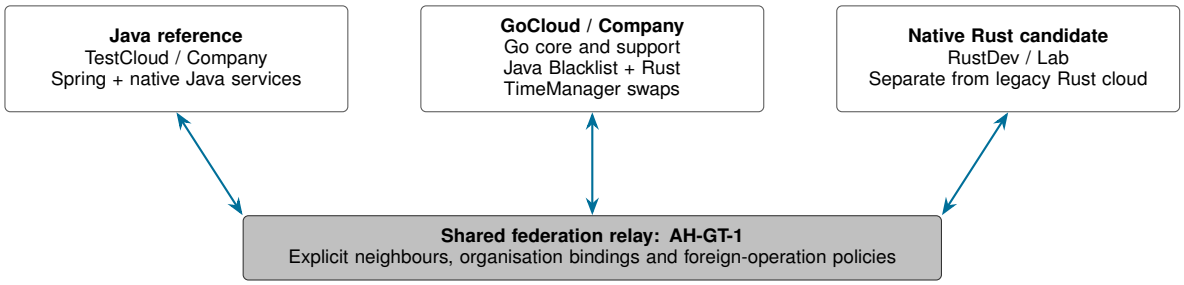


Figure 12: Implementation and trust boundaries. Federated clouds keep their own identities and state. GoCloud additionally demonstrates Java and Rust support providers under its own local-cloud identity and trust configuration.

10.1 Evidence and Current Deployment Scope

Five-core substitution (19–20 October). The Java-cloud interchangeability rerun placed Go and Rust implementations in the same five service slots as Java: ServiceRegistry, Authentication, ConsumerAuthorization, DynamicServiceOrchestration and SimpleStoreServiceOrchestration. The three transitions Go to Rust, Rust to Java and Java to Go passed the recorded core behavioural checks and the same ten Java support-client checks. DNS names, ports, cloud CA and system certificates were retained; each implementation kept separate persistent state. Corrections included the narrowly scoped identity bootstrap lookup, unrestricted bootstrap-discovery metadata, certificate-bound registration/discovery and Java-compatible identity service paths [20]. This dated sequence is evidence of the tested boundary, not an assertion about the currently selected image or database migration.

Native Rust federation and monitoring (19 October). The native `arrowhead-rust-candidate` deployment, identified as `RustDev|Lab`, is distinct from the older `arrowhead-v52-test-cloud-rust` deployment. Its native producer/historian path and AMQP–SysMonitor–MQTT–AIGym pipeline have functional evidence. AH-GT-1 global discovery, negotiation and encrypted HTTP forwarding were exercised with Java and Go peers, including historian read-back, operation/path denial and identity-binding checks. Cross-organisation exchange requires an explicit `RustDev=Lab` peer binding, approved neighbour records and scoped foreign-consumer policies; it does not merge cloud CAs, accounts or business data [19].

Plant-managed orchestration (23 October). GoCloud and the native Rust candidate now implement the Java project's *serviceOrchestrationPlantManagement* extension. Each passed 19 live extension checks, including atomic replacement, invalid-request rejection, persistence, priority/fallback behaviour, unavailable scopes and restriction under both inter-cloud flags. Both final snapshots were empty because no operator plant was active. The legacy Rust cloud has no PDE and was not part of this update. The consoles passed read checks; the report does not establish newly enabled console mutation rights [21].

Mixed-language and MQTT translation (24 October). GoCloud now exposes the Java-compatible MQTT API from all five Go core systems and registers `generic_mqtts`. Reversible swaps run the Java Blacklist with its MQTT API and a native Rust TimeManager with an HTTPS-only API against the Go core. The Go TranslationManager and **TestTranslation** exercise `ALLOW_TRANSLATION` with HTTPS/MQTT interface bridging and SenML-to-`simpleJson` data-model conversion. Recorded paths include MQTT consumption of the Go

DataManager, HTTPS consumption of the Go DSO MQTT API, direct/bridged calls to Java Blacklist, and MQTT consumption of Rust TimeManager through a bridge. Wrong bridge tokens, usage reporting and abort were checked; the reported regression passed both with the swaps active and after restoration [22].

Interpreting the source-contract audit. The 23 October audit compared 284 registered Java operations with native registrations and reported 256 exact Go matches and 229 Rust matches. Those are historical service/operation/method/path counts, not current equivalence scores: the 24 October MQTT/translation changes and newer runtime registrations were not re-audited here. The audit identified path-parameter conventions, renamed management operations, policy/version metadata and LocalCloudManager contract differences; DTO fields and error bodies were not measured. These remain acceptance dimensions rather than claims closed by the passing workload tests [23].

10.2 Contract and Security Requirements

A replacement must preserve the service definition and version, operation names, DTO fields and validation, interface-template properties, policy labels, discovery metadata and externally advertised base paths. HTTPS and MQTT are separate bindings: a compatible HTTPS endpoint alone does not imply MQTT compatibility. MQTT requests retain authentication, blacklist, management and authorisation checks, and responses are confined to the authenticated system's response topic. Identity login/logout/change remain HTTPS-only in the tested Go MQTT profile [22].

The broker certificate is also part of the cross-language contract. GoCloud now issues system-profile subjects for `AmqpBroker.GoCloud.Company.arrowhead.eu` and `MQTTBroker.GoCloud.Company.arrowhead.eu`, while retaining DNS/IP SANs. This satisfies Rust's explicit broker-identity pinning as well as Java/Go/Python hostname verification. Rust TimeManager publishes its native monitoring through AMQP with `messaging.amqpIdentity=AmqpBroker`; it remains an HTTPS-only application provider [22].

10.3 Boundaries and Acceptance of Further Replacements

The evidence establishes the recorded paths, not universal Java equivalence. TestTranslation is a test provider with bounded interface/data-model support; it does not demonstrate arbitrary translators or translation chains. The Rust candidate still lacks complete Java DTO/error/side-effect coverage and complete onboarding provisioning/renewal/off-boarding, advanced choreography, remaining QoS/translation/provider-MQTT integrations and full lifecycle qualification. Its segment-exact tunnel path matching is intentionally stricter than reference descendant matching. The 23 October PDE/DSO update supersedes the older Rust plant-synchronisation limitation, but does not close these other gaps [22, 19, 21].

For each further substitution, record the exact service operations and transports, implementation build, cloud identity, state store and policies. Verify positive consumer/provider calls and denial cases, including token/certificate mismatch, revocation on fresh connections, unauthorised operations and paths, broker topic isolation where applicable, persistence/restart behaviour and rollback to the previous implementation. Stop the replaced service before starting its successor to avoid duplicate identities, DNS aliases and ports. Java Blacklist has its own MySQL store and Rust TimeManager its own state volume; existing Go state is retained, not automatically converted. Passing these checks is functional acceptance within the recorded scope, not production or security certification.

For Go, remaining qualification includes QoS selection, complete push/recovery semantics, golden DTO/error fixtures and state migration. The current support compatibility notes still distinguish SysMonitor's MQTT 3.1.1 output from Java MQTT 5 metadata semantics; the new core MQTT API and translation tests do not by themselves close that telemetry binding gap. Conversely, older catalogue entries claiming no Go MQTT, no Go translation, no native monitoring, no federation or no Go CRL enforcement must not override newer code and dated evidence. The Go security source now defines fail-closed configured CRL checks, and native Rust still documents missing MQTT service APIs and remaining QoS/translation pipelines [9, 22, 19].

11 Implementation Observations and Open Issues

The following observations were made on the v5.2.2 sources and are proposed for discussion in the Roadmap WG:

1. **Framework version constant (fixed in workspace, v0.4).** `Constants.AH_FRAMEWORK_VERSION` in the released `arrowhead-common` 5.2.2 is `"5.2.1"`, so the released Core systems register themselves with system version 5.2.1. The workspace library sets it to `"5.2.2"`; systems built against it register 5.2.2. Upstream change proposed.
2. **Default port collision (resolved in v0.2).** **SimpleStoreServiceOrchestration**, **TranslationManager** and **Onboarding** all defaulted to port 8465. **SimpleStoreServiceOrchestration** now uses 8442 and **Onboarding** 8435, following the port numbering specification of Section 9. The upstream SimpleStore container image must still be updated (Table 10).
3. **Monitoring release alignment.** The older upstream *monitor* registration TODO is superseded in the deployed workspace by native *sysMonitor* AMQP offerings from all 22 microsystems. The project contract still requires upstream release alignment [18].
4. **Security-profile conformance.** Since v5.0 the default architecture uses full AA and encrypted information exchange. Component samples that disable TLS or select `DECLARED` are development exceptions. Packaging, startup and deployment checks must prevent their accidental use as the normal profile (Section 7.1).
5. **Atomic orchestration and restricted discovery.** With the default `restricted` discovery policy, consumers of **SimpleStoreServiceOrchestration** cannot resolve instance identifiers unless they are added to `service.discovery.direct.access`; the intended combination should be documented.
6. **Inter-cloud.** The orchestration interfaces anticipate inter-cloud orchestration but no Gatekeeper/Gateway is released for 5.2.2. The project implementations (Gatekeeper, GateTunnel, relay) exercise this path of **DynamicServiceOrchestration** in both directions between the test clouds; the provider-side Gatekeeper must use *authorizationManagement* `check-policies`, because *authorization* `verify` answers only the provider or consumer itself [17].
7. **User authentication** and authentication methods beyond `PASSWORD` and X.509 (e.g. OpenID Connect, OAuth) remain open, as in [2].

12 References

[1] J. Delsing, “Generic System-of-Systems Description (GSoSD): The Arrowhead Core and Support Systems 5.2.2, v2.1,” 2026, roadmap/5.2/Arrowhead SoS architecture/BlackBox_5.2.2/GSoSD-Core-and-Support-Systems-5.2.2.

[2] Eclipse Arrowhead Roadmap WG, “Generic System-of-Systems Description (GSoSD): The Arrowhead Core Systems 5.0, v1.2,” 2025, roadmap/5.2/Arrowhead SoS architecture/BlackBox_5.2.2/GSoSD Arrowhead Core Systems 5.0 v1.2.docx.

[3] EISLAB Arrowhead workspace, “Primary source policy and implementation inventories,” 2026, 30 October. Eclipse/PRIMARY-SOURCES.md; primary Java, Go and Rust README files. Local source designation, not an upstream release or automatic deployment.

[4] ——, “Independent Java support distribution verification,” 2026, eclipse-Arrowhead-java: JAVA-SUPPORT-FOLDERS.md and ALL-JAVA-SUPPORT-VERIFICATION-2026-10-25.md. Fifteen standalone projects; isolated builds/startup, not new full live-cloud acceptance.

[5] ——, “Go reorganisation verification,” 2026, eclipse-Arrowhead-go/verification/REPORT.md, 28 October. Twenty-two modules verified; optional native-registry integration failure retained. Later LocalCloudConfigurator is outside that audit.

[6] ——, “Rust workspace layout and compatibility boundaries,” 2026, eclipse-Arrowhead-rust/README.md; core and support COMPATIBILITY.md; LocalCloudManager/VERIFICATION.md. Five core entrypoints, fifteen shared-crate support binaries and a separate manager crate.

[7] ——, “Java LocalCloudConfigurator and graphical local-cloud editor,” 2026, eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java: README.md, documentation and verification/2026-10-30/REPORT.md. Baseline live workflow is verification-only.

[8] ——, “Integrated microsystem database assets,” 2026, java LocalCloudConfigurator verification/2026-10-30/database-assets/REPORT.md; catalog and runtime-isolation evidence. Sixteen private host-owned database services; 20 Java tests, ten focused manager/editor tests and ten browser groups.

[9] ——, “Native implementation compatibility and security boundaries,” 2026, sources inspected 24 October: ah5-core-go/docs/COMPATIBILITY.md and SECURITY.md; ah5-support-go/COMPATIBILITY.md; ah5-core-rust/COMPATIBILITY.md; ah5-support-rust/COMPATIBILITY.md. Later dated deployment evidence takes precedence over superseded gap lists.

[10] Eclipse Arrowhead, “ah5-core-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-core-java-spring, 2026.

[11] ——, “ah5-common-java-spring (arrowhead-common), version 5.2.2,” https://github.com/Aitia-IIOT/ah5-common-java-spring, 2026.

[12] ——, “ah5-blacklist-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-blacklist-java-spring, 2026.

[13] ——, “ah5-translation-manager-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-translation-manager-java-spring, 2026.

[14] ——, “ah5-device-qos-evaluator-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-device-qos-evaluator-java-spring, 2026.

[15] AITIA International, “Arrowhead Framework 5 – Java Spring implementation documentation (SysD, SD, IDD),” https://aitia-iiot.github.io/ah5-docs-java-spring/home/welcome/, 2026.

[16] Eclipse Arrowhead Roadmap WG, “Naming convention for local clouds, microsystems and microservices (Naming_convention_2),” https://github.com/eclipse-arrowhead/roadmap, 2025.

[17] J. Delsing, “Generic System of Systems Design Description: Arrowhead Support Systems 5.2 – White Box, v0.12,” 2026, roadmap/5.2/Arrowhead SoS architecture/WhiteBox_5.2.2.

[18] EISLAB Arrowhead workspace, “Java TestCloud deployment and orchestration snapshot,” 2026, 24 October. Arrowhead-test-cloud-java: compose.yaml, README.md and service-policy scripts; live Docker project arrowhead-v52-test-cloud, 22 microsystems and 41 containers.

[19] ——, “Native Rust federation and functional acceptance,” 2026, 19 October. Arrowhead-test-cloud-rust/candidate-native/FUNCTIONAL-FEDERATION.md and REPLACEMENT-STATUS.md; later PDE results supersede the earlier plant-synchronisation limitation.

[20] ——, “Interchangeability rerun, 19–20 October 2026,” 2026, arrowhead-test-cloud-java/interchangeability/RESULTS-2026-10-19.md. Five core slots, three interleaved implementation transitions and unchanged Java support clients.

[21] ——, “Go and native Rust PDE/DSO deployment,” 2026, 23 October. security-evaluation/pde-dso-go-rust-2026-10-23/README.md. Nineteen live extension checks passed in each cloud; console verification was read-only.

[22] ——, “GoCloud: MQTT API, translation bridges, Java and Rust support systems,” 2026, 24 October. Arrowhead-test-cloud-go/MQTT-2026-10-24.md; companion TRANSLATION-2026-10-24.md. Dated engineering test evidence.

[23] ——, “Arrowhead 5.2.2 Java, Go and Rust interchangeability,” 2026, 23 October. Local source-contract comparison, retained in the Interchangability analysis directory. Historical operation counts predate the 24 October MQTT/translation updates; DTO and error-body parity were not measured.

13 Revision History

13.1 Amendments

No.	Date	Rev.	Subject of Amendments	Author
1	2026-10-13	0.1	Initial white-box GSoSDD derived from the v5.2.2 Core implementation and its documentation.	Jerker Delsing
2	2026-10-13	0.2	Added the port numbering specification for local clouds and systems of local clouds (Section 9); resolved the 8465 port collision (SimpleStoreServiceOrchestration 8442, Onboarding 8435).	Jerker Delsing
3	2026-10-13	0.3	Port conformance table updated: remaining test-bed deviations fixed (DataManager 8461, TimeManager 8463, random-data-producer 8500, LocalCloudManager 8490/8491).	Jerker Delsing
4	2026-10-13	0.4	Framework version constant set to 5.2.2 in the workspace common library (Section 11).	Jerker Delsing
5	2026-10-15	0.5	Security model: credential provisioning and admission by Onboarding, and the PlantDescriptionEngine as management-layer client of the Core management services.	Jerker Delsing
6	2026-10-15	0.6	Inter-cloud orchestration realised over the relay (project implementations); port specification: GateTunnel ranges, AMQP broker for sysMonitor, relay, conformance; test-bed deviations fixed (GoCloud AMQP 6671, GateTunnel pool 8000–8099, Rust cloud moved to $k = 2$, 10xxx).	Jerker Delsing
7	2026-10-23	0.7	PDE backend migrated from SSO to DSO managed snapshots; updated dependencies, use cases, security constraints and migration requirements.	Codex (draft)
8	2026-10-24	0.8	Added Java deployment and service-path figures; scoped Java/Go/Rust interoperability evidence through 24 October, MQTT translation, native PDE/DSO and current deployment/security corrections.	Codex (draft)
9	2026-10-24	0.9	Applied the SysML v1.6 Papyrus library colour convention to all system diagrams, sequence participants and deployment graphics; added a shared palette and legend.	Codex (draft)
10	2026-10-24	1.0	Rewrote bootstrapping and corresponding security descriptions: full AA and encrypted information exchange are the default from v5.0; unsecured test modes are explicit exceptions.	Codex (draft)
11	2026-10-30	1.1	Updated primary Java/Go/Rust packaging and evidence limits, four-core configuration default, graphical engineering tools and integrated private database ownership.	Codex (draft)

13.2 Quality Assurance

No.	Date	Version	Approved by
1		0.6	